

实验指导 2 Linux 系统，用 PID 实施进程控制

同济大学计算机系 操作系统荣誉班 邓蓉

一、实验要求

包中 C 程序 test.c。每隔 2s，输出“Hello World! ”。 用 ctrl+c 结束这个程序的运行。

包中 C++程序 test2.cpp。每隔 2s，输出“Hello World! ”。 用 ctrl+c 结束这个程序的运行。

程序运行时，遵循 Unix、Linux 系统标准的信号语义：ctrl+c，向负责执行前台作业的所有进程发信号 SIGINT。用信号杀死所有进程。

作业 1：改写这两个程序（随便哪个）。完成 PPT 15 中的任务。

进程标识符 (PID)，是一个整数。是系统内部管理进程的一个索引



```
prompt> ./cpu A & ./cpu B & ./cpu C & ./cpu D &
[1] 7353
[2] 7354
[3] 7355
[4] 7356
A
B
D
C
A
B
D
C
A
C
B
D
...
```

系统派发4个新进程，执行这个作业

7353#进程：执行cpu程序，显示字符串“A”
7354#进程：执行cpu程序，显示字符串“B”
7355#进程：执行cpu程序，显示字符串“C”
7356#进程：执行cpu程序，显示字符串“D”

这些进程独立运行，互不干扰

小实验：另外开一个终端 (terminal)。

- 1、执行ps命令，看看系统正在运行的都是哪些进程。
- 2、在另一个终端，执行命令杀死输出 B字符串的进程。
kill -9 7354

操作系统
15921642146, drong2004@tongji.edu.cn

电信学院计算机系 邓蓉

15

有可能需要 ps -all
仔细观察 ps 命令的输出

二、4 个程序并发的命令行

1、cpu 程序 源码（要按需修改）

```
#include <stdio.h>
#include <unistd.h>
```

```
int main(int argc, char *argv[])
{
```

```

while(1)
{
    printf("Hello World! ..... ");
    sleep(2);
}
return 0;
}

```

2、编译：

命令行提示符> gcc -o cpu cpu.c

3、4 个进程并行

命令行提示符> ./cpu A & ./cpu B & ./cpu C & ./cpu D &

注意这样做不可以：

命令行提示符> ./cpu A & ;./cpu B & ;./cpu C & ;./cpu D &
这是因为： & 和 ; 不可以混用。

三、尝试使用云端 Ubuntu 系统。（自己有 Linux 系统的同学，忽略）：

按小时购买云服务器或云主机（Ubuntu 20.04）。云主机买回来直接可以用，免去了装虚拟机的麻烦。

买一台 1 核 1G 的云主机。调试好你的程序。仔细观察作业的输出。
scp，把你的源代码复制回本地。

Win10，开两个 PowerShell，远程登录你的云主机（**连它的外网 IP**）。每个 PowerShell 就是你云端 Ubuntu 系统（Linux 系统）的一个远程终端。

云主机的使用过程，请参看包中文件 云主机的使用.pptx。

做完实验，scp **（连它的外网 IP）**，把你的源代码复制回本地，交作业。

立即卖掉云主机!!! 持有云主机，即便它是关的，也会 24 小时向你收费。下次需要的时候，我们再买。

四、思考题：

包里的另一个程序 test.cpp，也能实现进程每隔 2s，输出“Hello World! ”。但是用 ctrl+c 无法结束这个程序的运行。（王子煦提供的代码，好极了。可以启发我们思考）

这个问题的解释，我们还需要学一些知识。问题大致出在 system() 函数。参看代码中的注释。